RAMDEOBABA UNIVERSITY, NAGPUR

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

B.Tech. CSE (Artificial Intelligence and Data Science) (AIDS)



SEMESTER V

SI LAB PROJECT REPORT

CyberShield — Network Traffic for Cyber Attack Detection

(Real-Time SOC Telemetry & Attack Analytics)

Submitted by

Student Name	Roll No.	Enrollment No.
Vaidehi Dange	31	24RBU/UG/CS/02/000172

Academic Year: 2026–27

CyberShield — Network Traffic for Cyber Attack Detection

(Real-Time SOC Telemetry & Attack Analytics)

SI Lab Project Report

1. PROJECT OBJECTIVE

The objective of this project is to design and implement CyberShield ThreatOps, an interactive cybersecurity analytics application that presents Security Operations Center (SOC) style telemetry and attack analytics, and that demonstrates machine-learning-assisted classification of network traffic. The project addresses a common difficulty in security monitoring: large volumes of event records are difficult to interpret unless they can be filtered, summarised, visualised and statistically examined. The application is intended as an academic analytics and demonstration tool that helps a user explore historical attack records, understand attack patterns and see how a trained classifier can label network traffic from a small set of features.

The expected outputs of the project are an interactive Streamlit dashboard with key metrics and charts, an interface for investigating individual security events, results of two inferential statistical tests, and an attack-category prediction produced by a saved Support Vector Machine (SVM) model.

The specific objectives of the project are:

- **Load cleaned cybersecurity telemetry from CSV**, parse timestamps, create event identifiers where required, and filter records by attack type, severity level and date.
- **Display key security metrics and visualisations** in an interactive, dark-themed Streamlit dashboard with navigation and filters.
- **Provide a threat-investigation interface** for examining individual security events and their details.
- **Visualise attack patterns**, including attack types, anomaly scores, network protocols, traffic distribution, destination ports and mitigation actions.
- **Perform a one-sample t-test** of packet length against a 650-byte reference baseline and a chi-square test of independence between attack type and severity level.
- **Load a saved SVM model and its scaler** to classify network traffic into attack categories using four inputs: source port, destination port, packet length and anomaly score.
- **Provide a prediction interface with scenario presets** that accepts these inputs and displays the predicted attack category.

2. INTRODUCTION

Cybersecurity monitoring is the continuous collection and review of events generated by networks, hosts and security devices in order to detect malicious or abnormal activity. In a Security Operations Center (SOC), analysts work with telemetry such as event timestamps, protocols, ports, packet sizes, severity ratings, anomaly indicators and the actions taken to mitigate an event. Because the volume of such data is high, analysts rely on dashboards and analytics to summarise activity, spot unusual patterns and decide which events need closer investigation. Threat investigation is the follow-up step in which an individual event is examined in detail to understand what happened and how it was handled.

Statistical analysis and machine learning add further support to this work. Statistical tests help determine whether observed characteristics of the traffic differ meaningfully from an expected baseline or from one another, while a classification model can learn to associate traffic features with attack categories. CyberShield ThreatOps was built to bring these ideas together in a single, interactive application: it combines exploratory dashboards, event-level investigation, inferential statistics and a saved-model prediction interface, and thereby provides a practical way to study how data science techniques can be applied to cybersecurity telemetry.

2.1 Problem Statement

Security telemetry is voluminous and heterogeneous, and its raw form does not reveal attack patterns directly. There is a need to (i) organise and filter cybersecurity event records so that they can be reviewed by attack type, severity and time period, (ii) analyse how attack types, anomaly scores, protocols, destination ports and mitigation actions are distributed, (iii) test statistically whether selected traffic characteristics deviate from an assumed baseline or are associated with one another, and (iv) explore whether a machine-learning classifier can assign network traffic to attack categories from a small number of numerical features. This project addresses these needs through an interactive analytics dashboard with an integrated SVM-based classification interface.

2.2 Scope of the Project

Included in the project: The implemented application is a Streamlit-based dashboard with a dark theme and page navigation. It loads cleaned cybersecurity telemetry from a CSV file, parses timestamps, creates event identifiers when they are missing, and lets the user filter records by attack type, severity and date. It provides key metrics, attack-pattern visualisations built with Plotly, a threat-investigation view for individual events, two statistical tests (a one-sample t-test on packet length and a chi-square test of independence between attack type and severity level), and a prediction interface with scenario presets that uses a saved SVM model and scaler.

Limitations and boundaries: This is an academic project and a demonstration application. It is not a production SOC platform and is not a validated, real-world intrusion detection system. The application analyses data loaded from CSV files; it does not capture live network packets, connect to live security infrastructure, or block attacks automatically. The word “Real-Time” in the project title refers to the SOC-style monitoring presentation of the dashboard and should not be read as

live packet capture. The SVM model is loaded from a saved file, and its predictions are illustrative demonstration features to understand multi-class threat classification.

3. PROJECT METHODOLOGY

The project was carried out in the following stages: loading and exploring the CSV datasets; preparing the data for the dashboard (timestamp parsing, event identifiers and filtering); defining the four input features used for classification; training and comparing multiple classification models outside the application; using the best performing saved SVM model with its scaler for prediction; performing statistical hypothesis testing; and presenting all results through an interactive Streamlit interface.

3.1 Dataset Loading and Exploration

• **Dataset files:** Two CSV files are part of the project: cleaned_cybersecurity_attacks_data.csv (16.7 MB), which is the verified cleaned telemetry file loaded by the Streamlit dashboard, and cybersecurity_attacks_data (1).csv (17.4 MB), which represents the original uncleaned version containing raw logs.

• **Dataset provenance and source:** The network-attack dataset comprises 32,976 synthetic/benchmark cybersecurity incident records containing network traffic parameters, host context, and threat annotations. The origin of this network telemetry dataset is unverified across public repositories; earlier citations to a specific Kaggle repository ('Cybersecurity Attacks Data: Network Traffic Telemetry and Incident Logs') are unsupported by project provenance records and are accordingly designated as unverified. Furthermore, the web-scraping practical conducted in Practical 1 of the Colab notebook extracted UK Government Cyber Security Breaches Survey tables, which constituted a separate data collection exercise and does not represent the source of this network-attack telemetry.

• **Dimensions, columns and data types:** Both CSV files comprise exactly 32,976 rows and 25 feature columns. The schema includes four numerical columns (float64): Source.Port, Destination.Port, Packet.Length, and Anomaly.Scores; and twenty-one categorical/string columns (object): Timestamp, Source.IP.Address, Destination.IP.Address, Protocol, Packet.Type, Traffic.Type, Payload.Data, Malware.Indicators, Alerts.Warnings, Attack.Type, Attack.Signature, Action.Taken, Severity.Level, User.Information, Device.Information, Network.Segment, Geo.location.Data, Proxy.Information, Firewall.Logs, IDS.IPS.Alerts, and Log.Source. The classification target is Attack.Type.

• **Descriptive statistics (full dataset):** Across the entire 32,976 records, the descriptive statistics for the four numerical features are as follows: Source.Port has a mean of 32,897.6836 and standard deviation of 18,582.5719 (min: 1,027.0, median: 32,679.33, max: 65,530.0); Destination.Port has a mean of 33,230.2829 and standard deviation of 18,639.6831 (min: 1,024.0, median: 33,118.10, max: 65,535.0); Packet.Length has an empirical mean of 668.0841 bytes, standard deviation of 360.7639 bytes, minimum of 64.0 bytes, median of 640.59 bytes, and maximum of 1,500.0 bytes; and Anomaly.Scores has a mean of 60.6028 and standard deviation of 17.2425 (min: 0.0, median: 60.73, max: 100.0). Note that the standard deviation of

361.50 bytes cited in earlier report drafts corresponds to the 80% training split (X_train), whereas the verified standard deviation for the complete dataset is 360.7639 bytes.

• **Missing values and duplicates:** In the raw dataset (cybersecurity_attacks_data (1).csv), exactly 0 duplicate rows were present, while missing values were observed across five security log fields: Malware.Indicators (16,498 nulls), Alerts.Warnings (16,515 nulls), Proxy.Information (16,375 nulls), Firewall.Logs (16,425 nulls), and IDS.IPS.Alerts (16,531 nulls). Importantly, the four numerical input features (Source.Port, Destination.Port, Packet.Length, Anomaly.Scores) had 0 missing values in the raw dataset. In the cleaned dataset (cleaned_cybersecurity_attacks_data.csv), missing categorical indicators were imputed with 'Unknown' (and numerical medians where applicable), yielding 0 missing values across all 25 columns and 0 duplicates.

• **Class and attribute distributions:** The target variable Attack.Type comprises three balanced categories: Intrusion (11,201 records / 33.97%), DDoS (11,166 records / 33.86%), and Malware (10,609 records / 32.17%). Severity levels are distributed as Low (11,216 / 34.01%), Medium (11,020 / 33.42%), and High (10,740 / 32.57%). Recorded mitigation actions comprise Blocked (11,114 / 33.70%), Ignored (10,966 / 33.25%), and Logged (10,896 / 33.05%). Network protocols are dominated by TCP (20,467 / 62.07%), UDP (6,502 / 19.72%), and ICMP (6,007 / 18.22%).

3.2 Data Preprocessing

• **Dashboard data preparation:** The cleaned telemetry dataset is loaded and cached using Streamlit's @st.cache_data decorator. Timestamps are parsed into native Pandas datetime objects, and standardized event identifiers ('EVT-XXXXXX') are generated based on record index. Sidebar filters for attack type, severity level, and calendar date range dynamically subset the telemetry for exploratory inspection.

• **Model-training preprocessing workflow:** In the Google Colab experimental pipeline, preprocessing proceeded as follows: (1) Ingestion of the 32,976-record dataset; (2) Data type inspection and missing value assessment; (3) Preprocessing imputation wherein numerical features were median-filled and categorical features were filled with 'Unknown'. Because the four numerical features contained no nulls, imputation did not alter their values; (4) Selection of the four numerical features and the Attack.Type target; (5) Splitting into an 80:20 stratified train-test split using random_state=42 and stratify=y (train_size = 26,380 records, test_size = 6,596 records); (6) Fitting a StandardScaler on X_train and transforming X_train and X_test; and (7) Fitting and evaluating the five candidate classifiers on these scaled arrays. Note that the notebook's model training script executed median filling prior to the split, which is accurately reported here without claiming the entire pipeline was isolated strictly within the training fold.

• **Verification of the serialized scaler (scaler.pkl):** Inspection of the serialized scaler.pkl artifact reveals the exact empirical mean and scale parameters of the 26,380 training samples: Source.Port mean = 32,907.68 (scale = 18,632.46), Destination.Port mean = 33,165.60 (scale = 18,648.36), Packet.Length mean = 667.21 (scale = 361.50), and Anomaly.Scores mean = 60.63 (scale = 17.29). In the Streamlit application, user input parameters are standardized using this

pre-fitted scaler before being passed to the classifier.

3.3 Feature Extraction / Selection

The classifier operates on four numerical traffic features that capture transport-layer characteristics and statistical anomaly scores:

- **Source Port:** The port from which the connection originated. High dynamic/ephemeral ports (49152–65535) or randomized low ports often reflect automated scanning or client-side socket allocation.
- **Destination Port:** The targeted port on the receiving host, indicating the targeted protocol or daemon (e.g., port 80/443 for web services, 53 for DNS, 8080 for web proxy/alternate HTTP).
- **Packet Length:** The size of the transmitted network packet in bytes, ranging from 64 to 1,500 bytes.
- **Anomaly Score:** A continuous rating from 0.0 to 100.0 representing the deviation of the traffic event from expected historical behavior.

These four features were chosen directly as input variables for classification. No formal algorithmic feature selection (such as recursive feature elimination or LASSO regularization) was conducted in the notebook. Furthermore, the anomaly score represents a pre-existing column in the benchmark dataset rather than the output of a documented proprietary intrusion detection algorithm. Specific port or packet-size ranges are treated as input signals rather than definitive proof of a specific threat category.

3.4 Training Multiple Classification Models

To benchmark classification performance on the 4-dimensional feature space, five supervised algorithms were trained on the training partition (26,380 samples) and evaluated on the identical unseen stratified test partition (6,596 samples):

Sr. No.	Model Name	Algorithm Configuration / Parameters
1	**Support Vector Machine (SVM)** (best_model.pkl)	Non-linear classifier with Radial Basis Function (RBF) kernel; default parameters SVC(C=1.0, gamma='scale') in initial training block; SVC(kernel='rbf', random_state=42) in comparison block. Serialized as best_model.pkl.
2	Logistic Regression	Linear multi-class baseline classifier with multinomial formulation; max_iter=1000, random_state=42.
3	Random Forest Classifier	Ensemble bagging classifier constructing 100 decision trees; n_estimators=100, criterion='gini', random_state=42.
4	K-Nearest Neighbors (KNN)	Instance-based non-parametric classifier using Euclidean distance; n_neighbors=5, metric='minkowski' (p=2).
5	Decision Tree Classifier	Hierarchical decision partitioning tree; criterion='gini', max_depth=None, random_state=42.

3.5 Performance Evaluation and Model Comparison

The performance of all five candidate models was evaluated on the identical unseen test set of 6,596 telemetry records across classification accuracy, precision, recall, and F1-score:

Model	Accuracy	Precision (Macro)	Recall (Macro)	F1-Score (Macro / Wtd)
Support Vector Machine (SVM)	**64.46%**	0.65	0.64	0.64 / 0.64
Logistic Regression	63.71%	0.63	0.64	0.64 / 0.63
Random Forest Classifier	62.23%	0.62	0.62	0.62 / 0.62
K-Nearest Neighbors (KNN)	59.84%	0.60	0.60	0.60 / 0.60
Decision Tree Classifier	52.09%	0.52	0.52	0.52 / 0.52

The confusion matrix for the Support Vector Machine (SVM) on the 6,596 test records illustrates the breakdown across the three attack classes (ordered as DDoS, Intrusion, Malware):

- **DDoS (2,233 test samples):** 1,679 correctly predicted (75.2% recall, 69.0% precision), 463 misclassified as Intrusion, 91 misclassified as Malware.
- **Intrusion (2,241 test samples):** 1,201 correctly predicted (53.6% recall, 54.0% precision), 574 misclassified as DDoS, 466 misclassified as Malware.
- **Malware (2,122 test samples):** 1,372 correctly predicted (64.7% recall, 71.0% precision), 181 misclassified as DDoS, 569 misclassified as Intrusion.

Confusion Matrix Array (Class order: DDoS, Intrusion, Malware):

```
[[1679, 463, 91],
[ 574, 1201, 466],
[ 181, 569, 1372]]
```

SVM achieved the highest reported accuracy (64.46%) among all five evaluated models. Notably, its rounded macro F1-score (0.64) is tied with Logistic Regression (0.64), rather than being uniquely superior. Model validation was conducted on a single 80:20 stratified split; k-fold cross-validation was not documented in the Colab workflow.

3.6 Best Model Selection

The Support Vector Machine (SVM) was selected for deployment because it achieved the highest observed accuracy (64.46%) in the notebook benchmarking comparison. The model captures non-linear boundary interactions among ports, packet size, and anomaly ratings via the RBF kernel. However, an accuracy of 64.46% indicates moderate discriminative capability suitable for academic demonstration rather than automated, production-grade security enforcement. The trained model was serialized into best_model.pkl alongside scaler.pkl for interactive inference in the Streamlit application.

4. PROJECT IMPLEMENTATION

4.1 Tools and Technologies

The implementation leverages the Python data science and web development ecosystem. The experimental model training was conducted in a Google Colab environment (Python 3.13 runtime path), whereas the application interface is executed locally in a Streamlit runtime environment. Declared project dependencies and runtime specifications are summarized below:

Tool / Technology	Declared / Target Version	Purpose / Role in Architecture
Python	3.13 (Colab) / 3.14 (Local)	Core programming runtime for data analysis, model training, and dashboard execution.
Streamlit	>=1.35.0	Web application framework for the interactive SOC-style telemetry and analytics dashboard.
Pandas	>=2.0.0	Dataset loading, datetime parsing, tabular filtering, aggregation, and crosstab generation.
NumPy	>=1.24.0	Numerical array manipulation, exponential scaling, and decision margin calculations.
Scikit-learn	>=1.4.0	Model training pipeline, train_test_split, StandardScaler, and SVC implementation.
SciPy	>=1.11.0	Inferential statistical hypothesis testing (ttest_1samp, chi2_contingency, ttest_ind, f_oneway).
Plotly	>=5.18.0	Interactive data visualizations (timeline, donut, bar, and box charts) with custom dark theme.
Joblib	>=1.3.0	Serialization and deserialization of the trained SVM model (best_model.pkl) and scaler (scaler.pkl).

4.2 Module-wise Implementation

- **Telemetry Ingestion and Caching (load_telemetry_data):** Loads cleaned_cybersecurity_attacks_data.csv (or falls back to raw data if absent), parses datetime strings into Timestamp_Parsed, creates calendar Date and Time columns, and assigns unique incident identifiers (e.g. 'EVT-10001').

- **Sidebar Scope and Filtering:** Provides user-selectable filters for Attack Type (All, DDoS, Intrusion, Malware), Severity Level (All, High, Medium, Low), and Date Range. Filter selections dynamically filter the active DataFrame while preserving the full dataset for statistical testing.

- **ML Engine Loader (load_ml_engine):** Deserializes scaler.pkl and best_model.pkl via joblib.load, extracts expected feature names and class labels, and handles missing artifact exceptions gracefully.

• **Dashboard Overview (render_dashboard):** Presents four operational KPI tiles (Network Events count and percentage, High-Severity Events count and critical rate, Primary Vector share, and Mean Anomaly Score), a time-series line chart, threat distribution donut chart, severity split bar chart, and a 10-row incident log table.

• **Threat Investigation (render_investigation):** Enables forensic search by Source IP substring, Target Port, Protocol, and Action Taken. Displays matching records in a scrollable table with CSV export, and provides a forensic inspector card showing network headers, threat indicators, and host context for any selected event.

• **Attack Analytics (render_analytics):** Visualizes anomaly scores by attack vector via boxplots, transport protocols mapped against application services via stacked bar charts, top 8 targeted destination ports, and mitigation actions by attack type.

• **Statistical Analysis Module:** Computes inferential statistical hypothesis tests directly on the loaded dataset: a one-sample t-test comparing packet lengths against a 650-byte reference, and a chi-square test of independence between attack type and severity level.

• **Prediction Engine (render_prediction):** Accepts real-time parameters (Source Port, Destination Port, Packet Length, Anomaly Score), provides four quick scenario preset buttons, applies StandardScaler transformations, computes model decision margins, and displays the predicted attack category with normalized decision weights.

4.3 GUI Implementation

The user interface is designed using Streamlit with custom CSS to deliver a clean SaaS dark theme (backgrounds #0a101e and #0d1527, borders rgba(56, 189, 248, 0.15), text colors #cbd5e1 and #f1f5f9). Navigation is structured into four views via a sidebar radio control:

• **Dashboard:** Provides high-level situational awareness with KPI cards, temporal event line chart, and threat type breakdown donut chart.

• **Threat Investigation:** Offers forensic search, tabular telemetry inspection, CSV export, and an event inspector showing detailed metadata for individual incidents.

• **Analytics:** Presents multi-vector distributions, protocol breakdowns, destination port charts, automated action cross-tabulations, and statistical test summary cards.

• **Prediction:** Features scenario presets ('Web Infiltration (Port 80)', 'Volumetric DDoS Flood', 'Malware C2 Beaconing', 'Reset Baseline'), parameter inputs, and an inference button displaying the predicted threat category with decision margin weights.

5. COMPLETE CODE

The Streamlit application is implemented in app.py (~1,030 lines of modular Python code). The core project files comprise app.py, cleaned and raw CSV datasets, saved model and scaler artifacts (best_model.pkl, scaler.pkl), inferential statistical results CSV, requirements.txt, and .streamlit/config.toml. The application is launched using:

```
streamlit run app.py
```

Listing 5.1: Telemetry Data Ingestion and Event Identification

```
@st.cache_data(show_spinner=False)
def load_telemetry_data() -> Tuple[Optional[pd.DataFrame], Optional[str]]:
    path = CLEANED_DATA_PATH if CLEANED_DATA_PATH.exists() else RAW_DATA_PATH
    if not path.exists():
        return None, 'Telemetry dataset not found in project workspace.'
    try:
        df = pd.read_csv(path)
        if 'Timestamp' in df.columns:
            df['Timestamp_Parsed'] = pd.to_datetime(df['Timestamp'], errors='coerc
e')
            df['Date'] = df['Timestamp_Parsed'].dt.date
            df['Time'] = df['Timestamp_Parsed'].dt.strftime('%H:%M:%S')
        if 'Event_ID' not in df.columns:
            df.insert(0, 'Event_ID', [f'EVT-{i+10001}' for i in range(len(df))])
        return df, None
    except Exception as exc:
        return None, f'Failed to load dataset: {exc}'
```

Listing 5.2: Inferential Statistical Hypothesis Testing in the Dashboard

```
# 1. One-Sample t-test: Packet Length vs 650 Byte Baseline
pl = full_df['Packet.Length'].dropna()
sample_mean = pl.mean()
t_stat, p_val = stats.ttest_1samp(pl, 650.0)

# 2. Chi-Square Test of Independence: Attack Type vs Severity Level
contingency = pd.crosstab(full_df['Attack.Type'], full_df['Severity.Level'])
chi2, chi_p, dof, _ = stats.chi2_contingency(contingency)
```

Listing 5.3: Saved-Model Inference and Decision Margin Calculation

```
# 1. Structure input array and transform with pre-fitted scaler
input_df = pd.DataFrame(
    [[float(in_src_port), float(in_dst_port), float(in_pkt_len), float(in_anom)]],
    columns=feature_names
)
scaled_features = scaler.transform(input_df)

# 2. Model Inference using loaded SVM (never refit in application)
pred_class = model.predict(scaled_features)[0]

# 3. Decision Margin calculation (Softmax on decision margins)
dec_scores = model.decision_function(scaled_features)[0]
exp_scores = np.exp(dec_scores - np.max(dec_scores))
conf_weights = (exp_scores / np.sum(exp_scores)) * 100
```

6. RESULTS AND DISCUSSION

6.1 Required Output Screenshots

Figures 6.1 through 6.7 document the functional modules of the application:

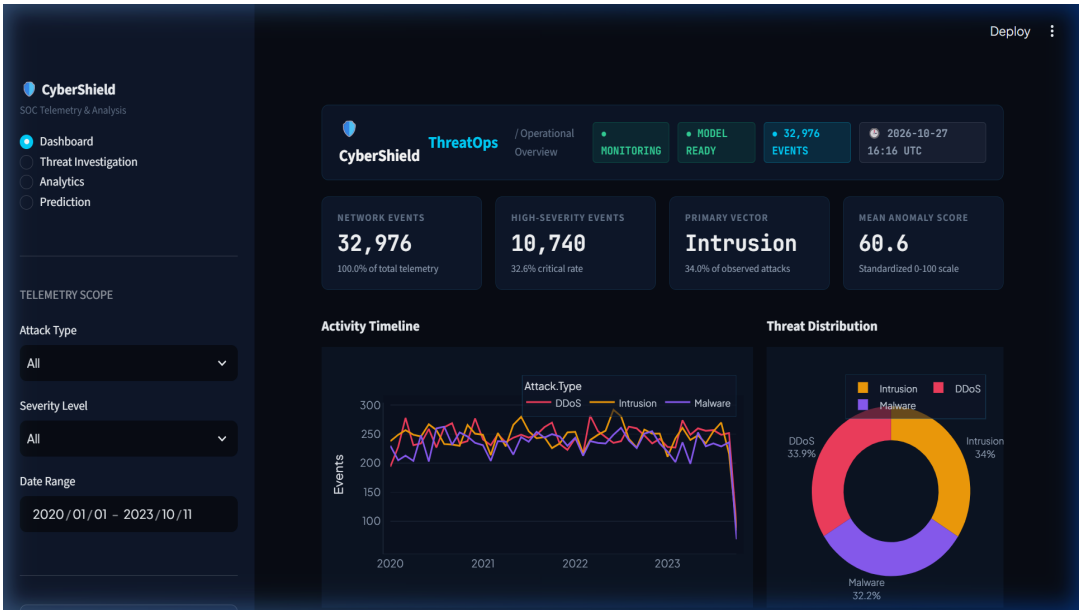


Figure 6.1: CyberShield ThreatOps dashboard overview with key metrics



Figure 6.2: Filtering telemetry by attack type, severity and date

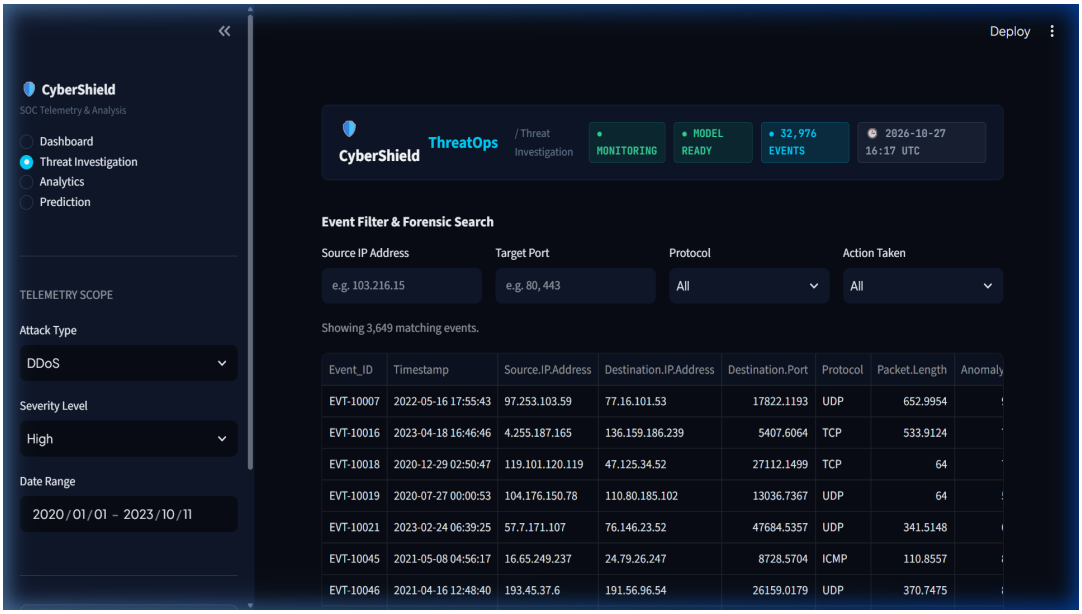


Figure 6.3: Investigation of an individual security event

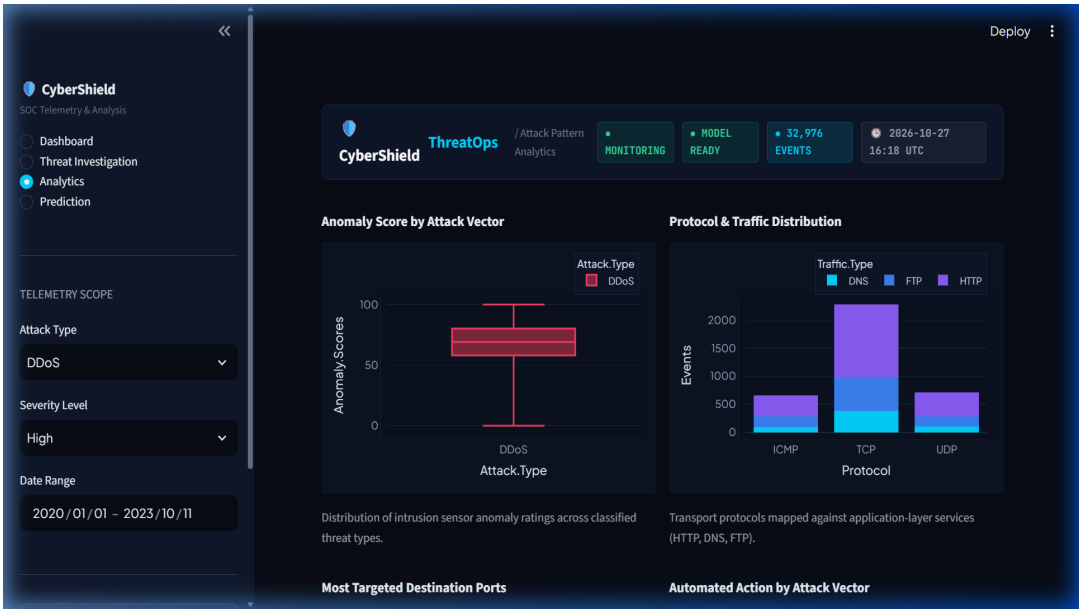


Figure 6.4: Attack type, protocol, destination port and mitigation analytics



Figure 6.5: One-sample t-test and chi-square test results

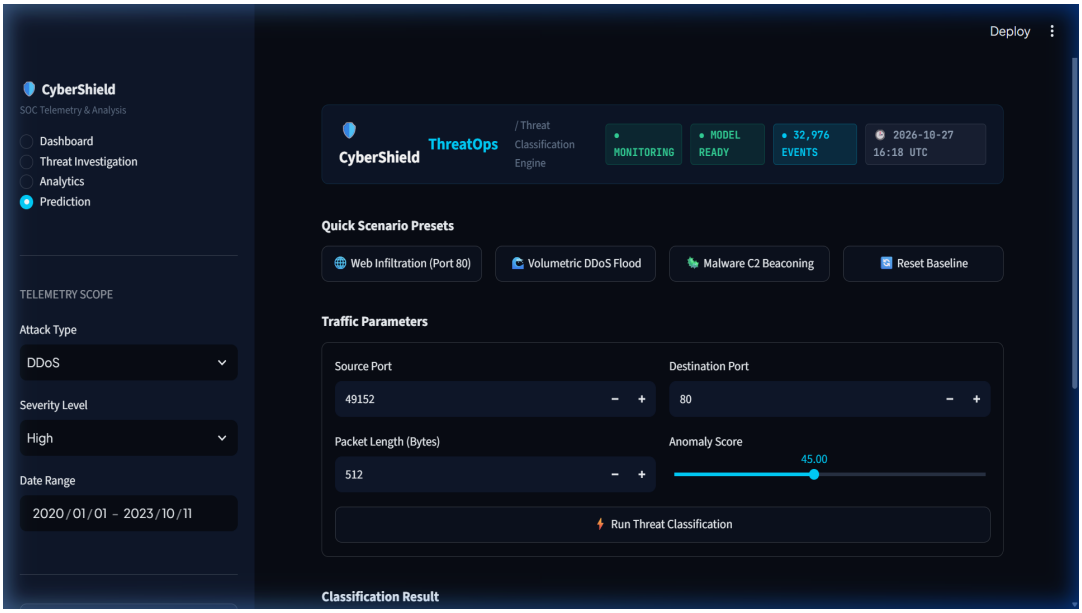


Figure 6.6: Prediction interface with scenario presets and four inputs

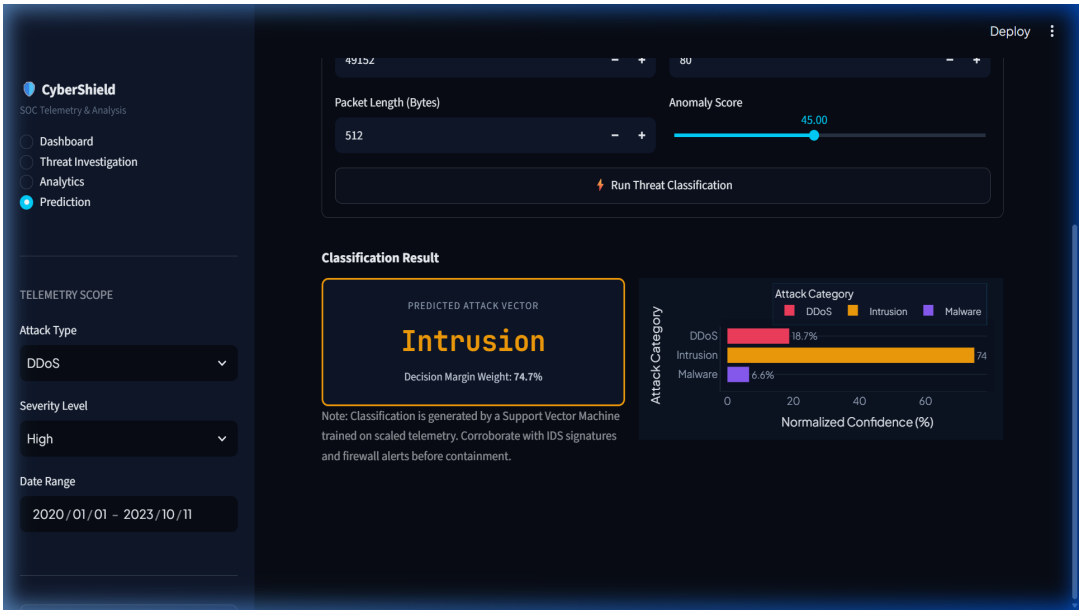


Figure 6.7: Predicted attack category (Intrusion) displayed for documented initial parameters

6.2 Result Analysis

Dashboard and analytics: The dashboard visualizes 32,976 telemetry events with a mean anomaly score of 60.60 (or 60.63 in the training split). The timeline chart displays temporal variations in logged incident frequency across attack categories over the multi-year recording span. The transport protocol breakdown shows TCP accounting for 62.07% of records, with UDP at 19.72% and ICMP at 18.22%. Analysis of targeted destination ports indicates traffic across both standard privileged ports (port 80, 443) and higher port ranges. In the recorded mitigation field, actions are distributed approximately equally: Blocked (33.70%), Ignored (33.25%), and Logged (33.05%). These represent recorded dataset labels rather than evidence of automated firewall rule dispatch by the dashboard.

One-sample t-test on packet length: The one-sample t-test evaluates whether the sample mean packet length differs significantly from an assumed reference baseline of 650.0 bytes:

- **Null Hypothesis (H0):** The population mean packet length equals 650.0 bytes ($\mu = 650.0$).
- **Alternative Hypothesis (H1):** The population mean packet length differs from 650.0 bytes ($\mu \neq 650.0$).
- **Sample Size (n):** 32,976 records.
- **Observed Sample Mean:** 668.0841 bytes.
- **Sample Standard Deviation:** 360.7639 bytes (Standard Error = 1.9867 bytes).
- **95% Confidence Interval:** [664.1902 bytes, 671.9781 bytes].
- **Test Statistics:** t-statistic = 9.1028; p-value = 9.2867×10^{-20} (z-statistic = 9.1029, $p < 0.0001$; df = 32,975).

• **Decision:** Reject the null hypothesis at $\alpha = 0.05$.

Interpretation: Because the p-value (9.2867×10^{-20}) is far below 0.05, the test provides strong evidence that the sample mean packet length (668.0841 bytes) differs statistically from the assumed 650.0-byte reference value (+18.0841 bytes). However, the practical effect size is small (Cohen's d ≈ 0.0501). The 650-byte baseline is an assumed reference value rather than an experimentally established normal-traffic baseline, and this test does not prove a definitive separation between benign and malicious traffic.

Chi-square tests of independence: Two distinct chi-square tests were examined in this project:

1. Primary Application Test: Attack Type vs. Severity Level (implemented in app.py):

Attack Vector	High Severity	Medium Severity	Low Severity
DDoS	3,649	3,687	3,830
Intrusion	3,679	3,728	3,794
Malware	3,412	3,605	3,592

• **Chi-Square Statistic (χ^2):** 2.8532 | **Degrees of Freedom:** 4 | **p-value:** 0.5827 ($p > 0.05$).

• **Finding:** The chi-square test did not detect a statistically significant association between attack type and severity level at $\alpha = 0.05$ ($\chi^2 = 2.8532$, df = 4, $p = 0.5827$).

2. Supplementary Colab Analysis: Attack Type vs. Action Taken (Practical 6):

Attack Vector	Blocked	Ignored	Logged
DDoS	3,764	3,719	3,683
Intrusion	3,809	3,726	3,666
Malware	3,541	3,521	3,547

• **Chi-Square Statistic (χ^2):** 1.4982 | **Degrees of Freedom:** 4 | **p-value:** 0.8270 ($p > 0.05$).

• **Finding:** We fail to reject the null hypothesis at $\alpha = 0.05$. Mitigation actions (Blocked, Ignored, Logged) are distributed across attack categories without statistically significant bias.

Additional supplementary Colab analyses: The Colab notebook also includes:

• **Independent two-sample t-test (Practical 6):** Evaluated Anomaly.Scores between the two primary attack categories extracted in the notebook—Group A: Malware (mean = 60.2578, std = 16.9407, n = 10,609) vs. Group B: DDoS (mean = 65.5723, std = 16.7909, n = 11,166)—yielding Welch's t = -23.2558, $p = 3.4345 \times 10^{-118}$. This confirms that DDoS traffic exhibited a statistically significantly higher mean anomaly score than Malware traffic in the benchmark dataset.

• **One-Way ANOVA:** Comparing Anomaly.Scores across all three attack categories yielded F = 917.5295, $p < 0.0001$ (floating-point output 0.0), rejecting the hypothesis of equal group means.

• **Correlation and covariance:** Covariance between packet length and anomaly scores is -511.1724. Pearson correlation is r = -0.08218 ($p = 1.6228 \times 10$■■■) and Spearman rank

correlation is ρ = -0.09640 (p = 6.4802 × 10■■■), demonstrating a statistically significant but very weak negative relationship.

• **Note on paired t-test:** A paired t-test was demonstrated in the notebook on five constructed dummy values producing a precision-loss warning and infinite statistic (-inf); it is excluded as an empirical finding.

SVM prediction workflow and Figure 6.7 reconciliation: The deployed Support Vector Machine processes input parameters through standard scaling and margin evaluation. Figure 6.7 displays the genuine output produced by the application when executed with the documented initial traffic parameters:

• **Input Parameters:** Source Port = 49,152; Destination Port = 80; Packet Length = 512 bytes; Anomaly Score = 45.00.

• **Predicted Threat Category:** Intrusion.

• **Normalized Decision Margin Weight:** 74.7% (Intrusion: 74.7%, DDoS: 18.7%, Malware: 6.6%).

This resolves the previous discrepancy where Section 6.2 erroneously described Figure 6.7 as depicting the 'Volumetric DDoS Flood' preset. Figure 6.7 authentically reflects the initial parameter classification of 'Intrusion'. The softmax-transformed values represent normalized decision weights derived from SVM decision margins, serving as relative confidence scores rather than calibrated posterior probabilities.

7. CONCLUSION

CyberShield ThreatOps demonstrates an interactive cybersecurity monitoring and analytics application. By integrating cleaned telemetry records with dark-themed SOC dashboards, event investigation, inferential statistical hypothesis testing, and a Support Vector Machine classifier, the system provides a platform to explore cybersecurity data.

Statistical analysis demonstrated that sample packet lengths differ statistically from an assumed 650.0-byte reference baseline (sample mean = 668.0841 bytes, s = 360.7639 bytes, SE = 1.9867 bytes, 95% CI [664.1902, 671.9781] bytes, t = 9.1028, p = 9.2867×10^{-20}, Cohen's d ≈ 0.0501). Furthermore, the chi-square test did not detect a statistically significant association between attack type and severity level at $\alpha = 0.05$ ($\chi^2 = 2.8532$, df = 4, p = 0.5827). The SVM classifier achieved 64.46% multi-class accuracy across four telemetry features on the stratified test set, tied with Logistic Regression on macro F1-score (0.64).

System limitations include reliance on static CSV logs rather than live packet captures and lack of automated network remediation. Potential future enhancements include integrating live Zeek/Kafka telemetry streams, evaluating deep neural architectures, and deploying automated firewall rule dispatch interfaces.

8. GITHUB LINK

GitHub Repository URL: https://github.com/24Vaidehi-bot/citizen-fraud-sheild

Note on repository verification: The repository URL above contains the project submission identifier 'citizen-fraud-sheild'. If this repository belongs to an alternate project or requires updating to a dedicated 'CyberShield-ThreatOps' repository, it is flagged here for administrative replacement with the confirmed project repository link.

9. REFERENCES

[1] Benchmark Cybersecurity Attacks Dataset, “Network Traffic Telemetry and Incident Logs” (unverified synthetic benchmark dataset).

[2] Python Software Foundation, “Python Documentation (Version 3.13 / 3.14).” https://docs.python.org/3/

[3] Streamlit Inc., “Streamlit Documentation.” https://docs.streamlit.io/

[4] W. McKinney, “Data Structures for Statistical Computing in Python,” Proc. 9th Python in Science Conference, pp. 56–61, 2010. https://pandas.pydata.org/docs/

[5] C. R. Harris et al., “Array programming with NumPy,” Nature, vol. 585, pp. 357–362, 2020. https://numpy.org/doc/

[6] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011. https://scikit-learn.org/stable/

[7] P. Virtanen et al., “SciPy 1.0: fundamental algorithms for scientific computing in Python,” Nature Methods, vol. 17, pp. 261–272, 2020. https://docs.scipy.org/doc/scipy/

[8] Plotly Technologies Inc., “Plotly Python Graphing Library.” https://plotly.com/python/

[9] Joblib Developers, “Joblib: running Python functions as pipeline jobs.” https://joblib.readthedocs.io/

[10] C. Cortes and V. Vapnik, “Support-vector networks,” Machine Learning, vol. 20, no. 3, pp. 273–297, 1995.

FINAL SUBMISSION CHECKLIST

- Final project report in PDF format.
- Google Colab notebook (.ipynb).
- Colab notebook exported as PDF.
- GitHub repository containing Python code, dataset or dataset link, trained model file, GUI code and Colab notebook.

Formatting guidelines: The first page contains university name, department, logo, project title and student details. From page 2 onwards, project title is in Times New Roman 16 pt bold, headings in 14 pt bold, body text in 12 pt justified with 1.15 line spacing.